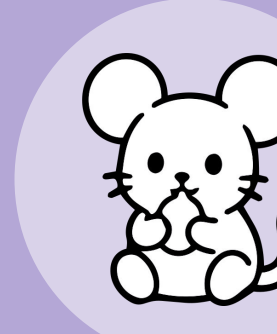


Figgy: Interception and Modification of Compilation Commands Across Build Systems



SecclaBU

Amanda Li^{1,2}, Khosro Moeini², Gianluca Stringhini², Manuel Egele²
The Bear Creek School, Redmond, WA USA¹; Boston University, Boston, MA USA²
wa.amandali@gmail.com



MOTIVATION

Coverage-guided fuzzing, compilation-database generation, and LLVM bitcode extraction all depend on two shared capabilities: **observing** every compiler command a build system issues, and **modifying** that command before it executes.

Problem:

- Build systems are supposed to respect environment conventions like CC/CXX/CFLAGS, but many hardcode compiler paths, invoke compilers through subprocesses that ignore modified environments, or use build-system-specific overrides that do not match these conventions.
 - Blight - intercepts by rewriting \$PATH so cc/gcc/clang resolve to its wrapper.
 - Bear - intercepts by using LD_PRELOAD to hook exec-family libc calls.
 - Both methods occur above the syscall layer, where build systems have common ways to route around interception.
- Tools like strace or eBPF can observe the syscall boundary but are restricted to read-only logging

AIM: Figgy's goal is a **general-purpose** interception system that **both traces** and **modifies execve(2) syscalls** at the syscall **boundary** so that compiler commands can be observed and rewritten regardless of build-system behavior, and exposed through a programmable module interface reusable across different use cases.

METHODS

Definitions

- execve:** Linux system call (syscall) that starts a new program and replaces the current process image
- ptrace:** Linux tracing/debugging interface that lets one process observe and control another
- sbuid:** a Debian tool that compiles packages inside isolated environments
- Wall-clock time:** Total elapsed time from the start to end of a run (includes CPU time, I/O, and scheduling delays)
- Context switch:** Transition where CPU switches from executing user-space code to kernel-space code (or back)
- seccomp-BPF:** Linux mechanism that filters syscalls using a small Berkeley Packet Filter (BPF) program

Tracing Pipeline

- Fork and execute the traced process:** follow all subsequent fork/clone events a tracee issues
- Parse execve arguments:** (path, argv, envp) at syscall entry, before the kernel resolves or runs them
- Optionally modify the execve arguments**
 - modify** - rewrites the original command in place
 - nomodify** - duplicates the command so the user's original execve runs unmodified, while a copy is ran separately
 - nomodify-drop** - the user program issues its execve, but the original call is dropped rather than executed, leaving only the substitute command

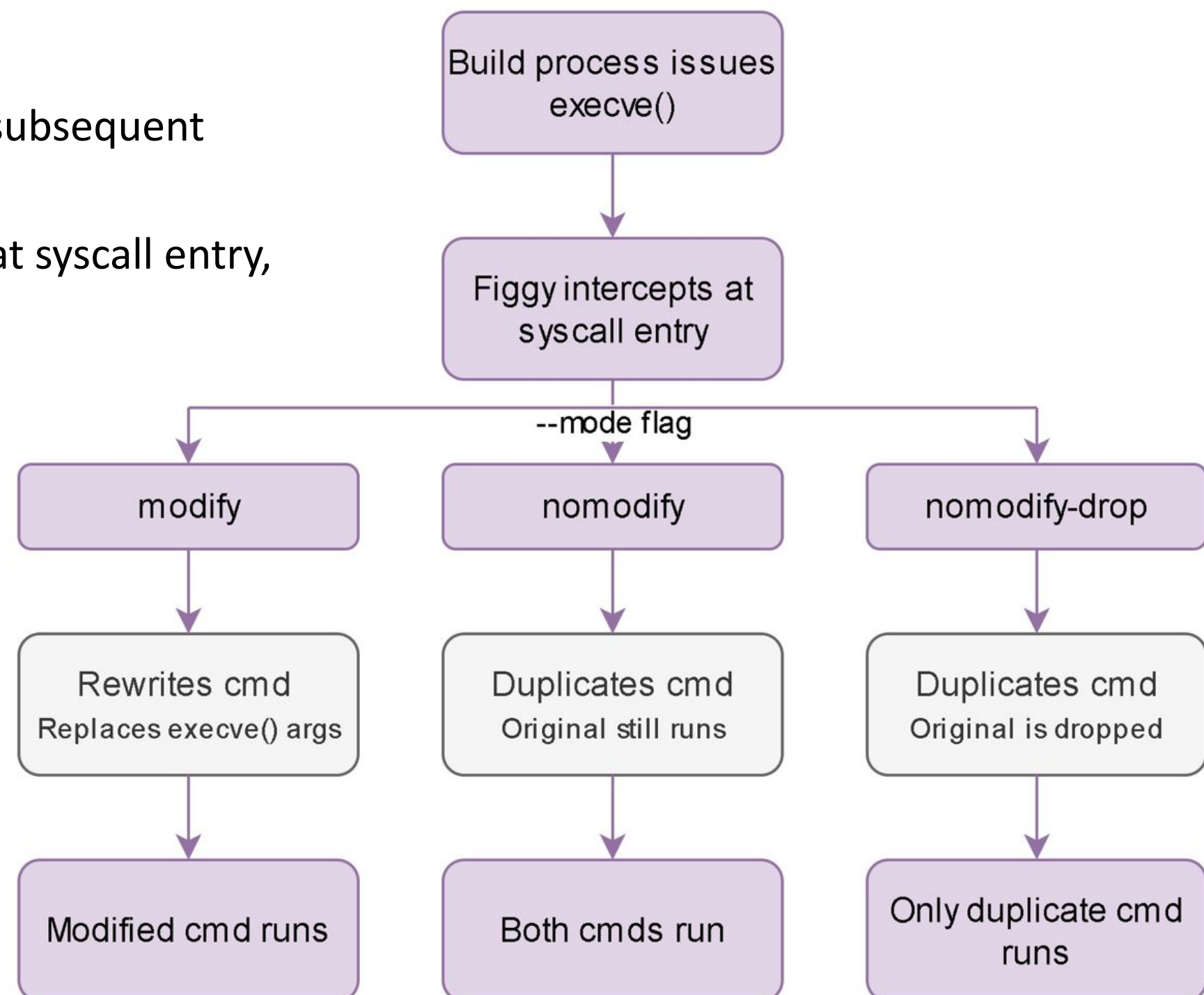


Figure 1. execve interception modes

Module Interface

- User-defined transformations** are written as Go **plugins** against a small built-in **interface**:
 - Module Name: Name() string
 - Modified Exec Call: Transform(call ExecCall) (ExecCall, bool)
- Modules are compiled with make <module_name> and loaded at runtime via --module
 - For each intercepted execve(), Figgy passes the call through the active module(s) and applies the returned transformation according to the selected mode

Use Case 1: Fuzzing Instrumentation (AFL)

- Packages are built with **sbuid**
 - ccswap module** substitutes the system's native C compiler with afl-clang-fast/afl-clang-fast++
 - fuzz_pkg.sh** contains the end-to-end sequence of commands needed to build, instrument, and fuzz a package with AFL

Use Case 2: LLVM Bitcode Generation

- Programmed a module (**argvinject**) to append compiler flags to Makefile execve() commands
- Flags Injected:** -flto=full -ffat-lto-objects -Wl,-mllvm,-lto-embed-bitcode=optimized

Further Optimization: Seccomp-BPF Filtering

- Seccomp-BPF Filtering (call through --seccomp) is an optional feature implemented to minimize program overhead
 - With normal ptrace, four context switches occur between the user and the kernel for every syscall
 - A seccomp-BPF filter sits at the kernel level, and filters for execve syscalls to send to the tracer so only execve syscalls have four context switches while all other syscalls run at native speed with two switches

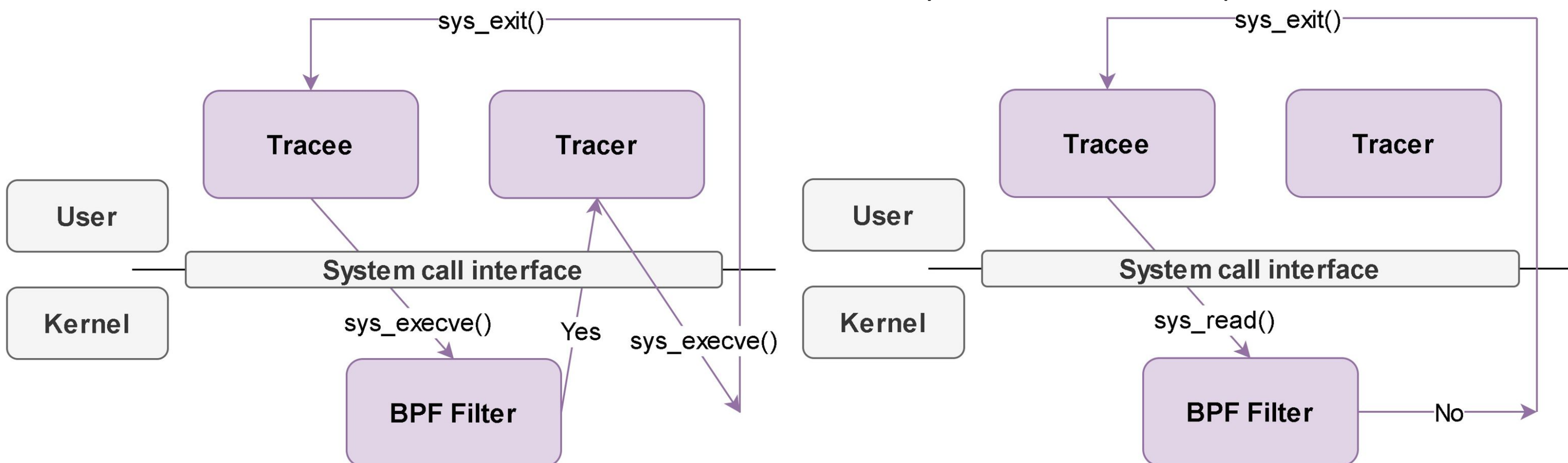


Figure 3. Standard ptrace tracing (left) vs. Seccomp-BPF filtered tracing (right)

RESULTS

Compiler invocation extraction and AFL fuzzing instrumentation

Figgy was run against **9 Debian packages** to extract compiler commands. For each package, successful capture of all execve syscalls, unmodified instrumentation, and successful fuzzing with AFL were recorded.

LLVM Bitcode Generation

To verify that Figgy's argvinject module successfully produces embedded LLVM bitcode, **readelf -S <package_name>** was ran against each build output and the presence of the **.llvmbc section** was checked. A package was **marked successful** if **llvmbc** appeared in the section headers.

	Invocation Extracted	Build Packages	Instrumented Unmodified	Fuzzed with AFL	.llvmbc present
openssl	✓	✓	✗	✗	✓
ffmpeg	✓	✓	✗	✗	✓
gzip	✓	✓	✓	✓	✓
tcpdump	✓	✓	✓	✓	✓
rolldice	✓	✓	✓	✓	✓
tar	✓	✓	✓	✓	✓
bzip2	✓	✓	✗	✗	✓
busybox	✓	✓	✓	✓	✓
sqlite3	✓	✓	✗	✗	✓

Table 1. Compiler invocation extraction and AFL instrumentation, Compilation database generation, and LLVM bitcode generation success rates across 9 Debian packages built via sbuid

seccomp-BPF Program Overhead Evaluation

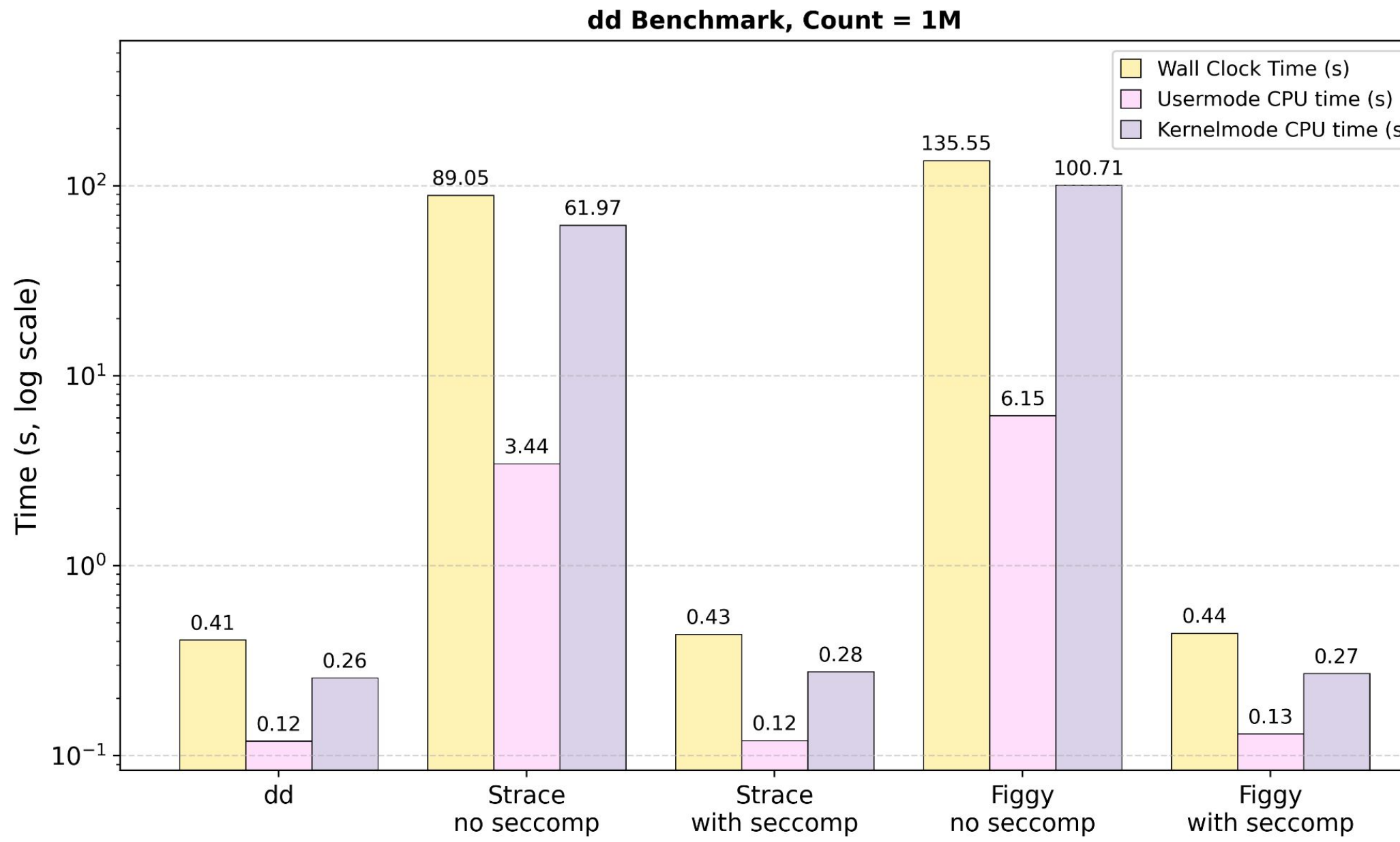


Figure 4. dd benchmark - values are averaged across 3 runs

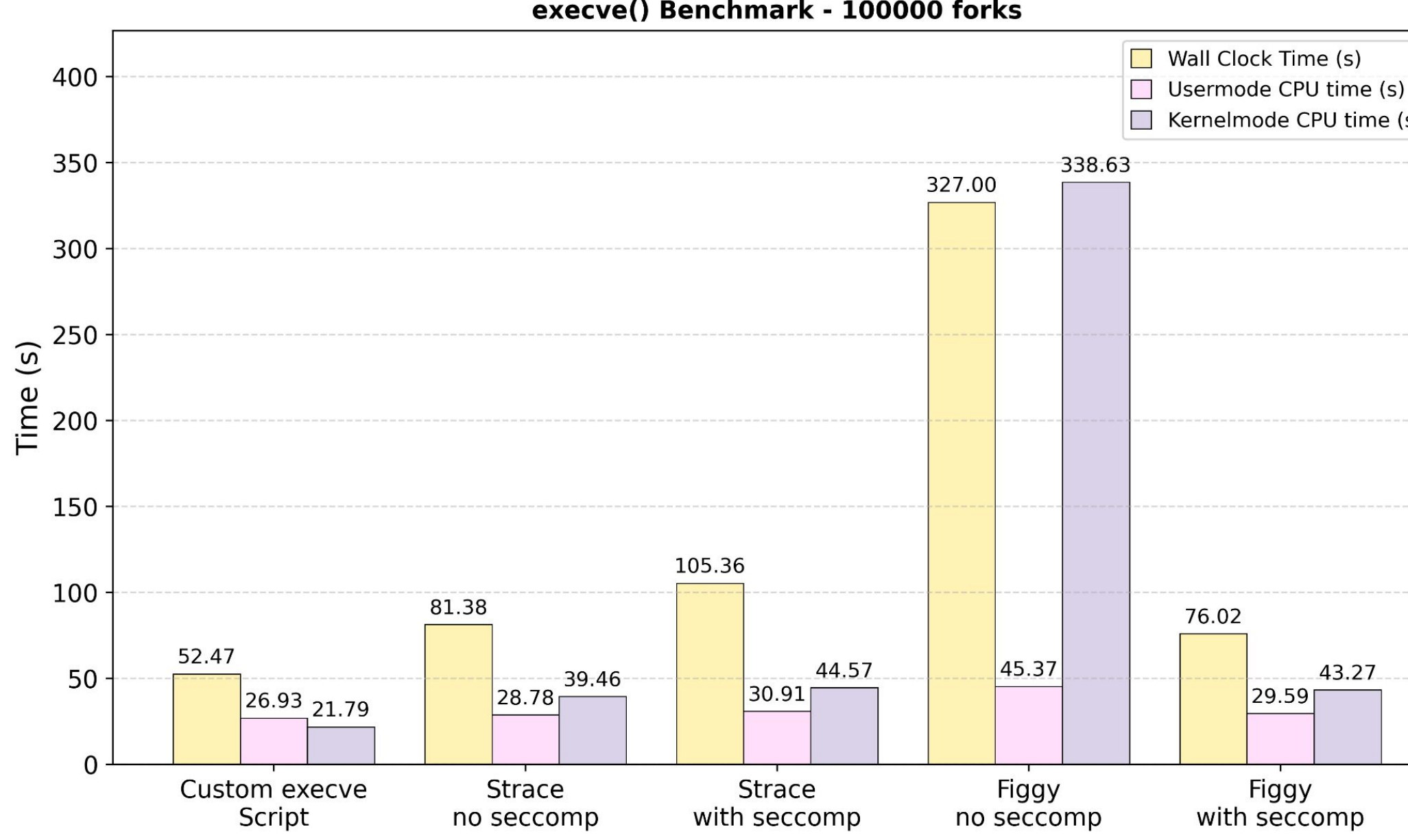


Figure 5. execve benchmark - values are averaged across 3 runs

- We measured **wall-clock time** by running **two workloads**: a syscall-heavy baseline and a custom execve loop

	Bytes per second
Count = 1M	
dd	2.6 MB/s
Strace no seccomp	12 kB/s
Strace with seccomp	2.5 MB/s
Figgy no seccomp	7.8 kB/s
Figgy with seccomp	2.5 MB/s

Table 2. Throughput of dd across tracing configurations.

Across both benchmarks, **seccomp-BPF consistently** brought Figgy's **runtime close** to the **untraced baseline**, indicating that most of ptrace's overhead can be eliminated by shifting tracing logic into in-kernel filtering.

CONCLUSION

Conclusion

The results suggest that **syscall-level execve interception** is a **reliable mechanism** for **observing** and **modifying** compiler commands without per-package customization.

- Figgy demonstrates that a **single interception engine** can **generalize** across **multiple use cases**, including tracing build commands, security monitoring, and improving debugging visibility, as **reusable infrastructure** rather than as being resolved independently by each tool.
- Figgy **successfully captures compilation data** under sbuid's choct (schroot) build environment, providing a seamless **tracking capability** that existing tools like strace, blight, and bear cannot achieve without complex manual configuration due to container isolation boundaries

The **seccomp-BPF results** indicate that the **overhead** typically associated with ptrace-based tracing is a consequence of naive tracing that **stops on every syscall**.

- By preselecting **only execve** at the kernel level, Figgy **retains** the **reliability** of syscall-boundary interception while **avoiding** the **cost** of tracing syscalls that are **irrelevant** to **compiler command tracking** for Figgy's purpose.

Future Improvements

- setuid Compatibility:** Address compatibility issues between seccomp filtering and setuid-dependent modes in build tools like sbuid's unshare mode.
- Accessibility for non-expert users:** The module interface requires users to write and compile Go code against Name()/Transform(), which assumes a level of programming familiarity the average person may not have.

ACKNOWLEDGEMENTS

I would like to thank Professor Gianluca Stringhini for his guidance and insights throughout this project. I also thank Professor Manuel Egele for his thoughtful advice on design choices, which greatly strengthened this work. Finally, would like to thank my main mentor, Khosro Moeini, for his direct involvement, continued support, conceptual explanations, and design suggestions, all of which were vital to the success of this project.